# SYSTEM MODIFICATION PROMPT — WEB-BASED BARANGAY COMPLAINT MANAGEMENT SYSTEM WITH CHATBOT INTEGRATION

## 1. PRIMARY INSTRUCTIONS — READ BEFORE IMPLEMENTING

I want you to modify my existing **Web-Based Barangay Complaint Management System with Chatbot Integration**. I have already developed the system, and its current files, components, designs, backend, database, and features must be preserved.

**STRICT RULES:**

- DO NOT start from scratch or rebuild the application.
- DO NOT replace my existing architecture, framework, database, or backend.
- DO NOT redesign unrelated pages, sections, components, or features.
- DO NOT remove existing functionality unless explicitly instructed below.
- DO NOT modify anything outside the scope of the requested changes.
- DO NOT introduce unnecessary libraries, dependencies, fonts, colors, or design systems.
- Inspect the existing codebase before making changes. Identify the relevant files, components, database tables, API endpoints, and current workflows.
- Reuse existing components, functions, API endpoints, authentication, database structures, and styling wherever possible.
- If a database or API change is necessary, make the smallest compatible modification and preserve existing records.
- Do not create duplicate features or parallel implementations when an existing feature can be extended.
- Maintain the current visual identity and responsive layout of the application.
- Do not use mock data, hardcoded results, or simulated interactions as substitutes for real functionality.
- Test every modification and fix any regressions caused by the changes.

Implement only the six modification groups described below. If a requirement conflicts with existing functionality or requires a technical decision, inspect the current implementation and choose the smallest safe, compatible solution.

---

# 2. COMPLAINANT INFORMATION, ANONYMITY, AND PRIVACY

## A. Add an anonymity selection in the complaint form

In the resident/user interface, locate Step 3: Complainant Information.

Add a clear selection that allows the complainant to choose between:

1. **Use My Real Name**
2. **Submit Anonymously**

The real-name option must use the name associated with the authenticated user's existing profile or registration information. Do not require the resident to manually enter a name that is already available in the profile.

If the resident selects anonymous submission, the complaint must be recorded and displayed as anonymous wherever the complainant's identity is presented in the complaint interface.

## B. Apply anonymity consistently in the admin interface

In the admin complaint information and case details:

- When the complainant selected "Use My Real Name," display the complainant's name according to the existing permissions and privacy rules.
- When the complainant selected "Submit Anonymously," display **Anonymous** instead of the user's actual name.
- Never reveal the real name through complaint details, lists, exports, generated reports, or other resident-facing or staff-facing views where anonymity is required.
- Do not expose the original name through frontend responses, browser-visible data, or API payloads that do not need it.

Preserve the internal complaint-to-user relationship when needed for authorized processing, notifications, and case management. Anonymity must not break complaint tracking or administrative workflows.

## C. Remove the Person Involved email field

Remove the **Person Involved Email** field from the complaint form and its related presentation, validation, and processing logic.

Keep the **Complainant Email** field in Step 3 as required by the existing workflow.

## D. Mask email addresses in the admin interface

In admin-facing complaint details and user information, mask email addresses so that only the first two characters of the email's local part remain visible, followed by asterisks and the original domain.

Example:

```
ri**********8@gmail.com
```

The example illustrates the desired masking style. Implement a consistent masking function that preserves the domain and conceals the remaining local-part characters. Avoid exposing the original email through the rendered interface or unnecessary client-side data.

Apply the masking consistently to relevant admin-facing views, tables, and complaint details. Do not unintentionally alter email addresses used internally for authorized notifications or account operations.

### E. Validation and testing

Verify that:

- Both anonymity options work.
- The default real name comes from the authenticated user's profile.
- Anonymous complaints never display the actual complainant name in unauthorized views.
- The complainant email remains available in Step 3.
- The Person Involved Email field is removed.
- Admin-facing emails are masked consistently.
- Existing complaint records and unrelated user information remain intact.

---

# 3. INCIDENT LOCATION SEARCH, BARANGAY VALIDATION, AND HEATMAP INTEGRATION

### A. Replace the existing incident-location form structure

Locate the incident-location step in the complaint submission form.

Remove the existing **San Andres Area** selection and the existing Leaflet map from this form, as explicitly requested.

Replace the current location-entry structure with a functional address search input.

The location input should allow residents to search for streets, subdivisions, areas, landmarks, and other relevant addresses. Provide address suggestions while the user types, similar to the search experience in Google Maps.

Do not merely change the appearance of the input. Implement actual address lookup and selection.

## B. Integrate a suitable geocoding or places-search provider

Inspect the current project and determine whether an existing mapping service or location API can be reused.

If none is suitable, select a compatible address-search or geocoding provider. You may use an external service, but you must integrate and test it properly.

Requirements:

- Show relevant address suggestions as the user types.
- Allow the user to select a suggested result.
- Retrieve the selected address and its geographic coordinates when supported.
- Handle loading states, empty results, API failures, and invalid selections.
- Debounce search requests where appropriate.
- Secure API credentials and follow the provider's restrictions.
- Avoid unnecessary API requests and duplicate geocoding calls.
- Preserve the exact selected address rather than replacing it with an approximate or shortened version.

Do not claim that the integration is functional until the real search and selection workflow has been tested.

## C. Validate whether the incident is within Barangay San Andres

After the resident selects an address, verify that its geographic location falls within the actual administrative boundary of **Barangay San Andres, Cainta, Rizal**.

Do not validate based only on whether the address text contains "San Andres." Use geographic coordinates and a reliable barangay boundary where available.

If the selected address is outside Barangay San Andres:

- Clearly inform the resident that the location is outside the barangay's coverage.
- Prevent the resident from proceeding to the next step.
- Require the resident to select a valid location within Barangay San Andres.

If the boundary data is unavailable or the location cannot be verified, do not silently accept the address as valid. Provide an appropriate error or verification message.

Perform validation on the backend as well as in the frontend wherever the existing architecture permits. Do not rely exclusively on client-side validation.

## D. Integrate selected addresses with existing admin heatmap areas

The admin interface already contains configured areas used for complaint heatmap reporting. Reuse these existing areas instead of creating a duplicate area-management system.

When a resident selects a valid incident location:

1. Retrieve or determine its geographic coordinates.
2. Match the coordinates to the appropriate configured admin heatmap area.
3. Associate the complaint with the matching area.
4. Save the relevant location and area association.
5. Ensure the complaint is reflected in the correct area in the admin heatmap.

For example, if the selected location belongs to **Eastbank Floodway**, the complaint must be associated with Eastbank Floodway and included in the corresponding heatmap results.

Use the actual area boundaries or mapping rules already configured in the system. Do not hardcode Eastbank Floodway or other area assignments based solely on address-name matching.

If an address is within Barangay San Andres but does not fall within any configured heatmap area, preserve the complaint's exact location and handle the unmatched area without assigning it to an incorrect area.

## E. Preserve manual map pinning

Although the existing Leaflet map must be removed from the complaint form as instructed, residents must still have the ability to manually pin the incident location.

Implement or reuse a compatible map-based pinning interaction that works with the new address-search workflow.

The resident should be able to:

- Search for an address.
- Select a suggested address.
- Adjust the location manually using a map pin when necessary.
- Confirm the final incident location.
- Have the final coordinates validated against the Barangay San Andres boundary.
- Have the confirmed location reflected in the admin heatmap.

If the user adjusts the pin, ensure the coordinates and saved address remain consistent. Revalidate the final pin location before allowing submission.

Do not simply restore the old Leaflet map unchanged. Reuse or implement the map interaction in a way that fits the new workflow.

### F. Retain the exact address

Preserve the exact address entered or selected by the resident for further details, even when the map's coordinates are not perfectly accurate.

Store the address separately from geographic coordinates when necessary. Do not overwrite the exact address with a reverse-geocoded approximation.

If manual pinning changes the coordinates, keep the resident's confirmed address available for reference and case documentation.

### G. Remove the duplicate existing pin feature

The existing "Pin Incident Location" feature is being addressed through this new location workflow.

Remove or consolidate the old duplicate implementation so that there is only one coherent incident-location workflow. Preserve any required underlying functionality and avoid breaking existing complaint records.

### H. Location testing requirements

Test the following cases:

- A valid street address within Barangay San Andres.
- A valid subdivision or area within the barangay.
- An address outside Barangay San Andres.
- An address that cannot be found.
- A location that cannot be verified because of a provider or boundary-data failure.
- A valid address with inaccurate map positioning.
- A manually adjusted pin within the barangay.
- A manually adjusted pin outside the barangay.
- A complaint within Eastbank Floodway or another existing heatmap area.
- A valid barangay location that does not match any configured heatmap area.

Confirm that the saved address, coordinates, complaint record, and heatmap association remain consistent.

---

# 4. CHATBOT FUNCTIONALITY AND INTERACTIVE ASSISTANCE

## A. Improve the existing chatbot

Extend the current chatbot rather than replacing it with a separate chatbot application.

The chatbot must provide actionable assistance instead of only answering questions and displaying informational text.

Preserve its existing interface and integrate the new behavior with the system's actual pages, authentication, complaint records, and admin messaging functionality.

## B. Account registration assistance

When a resident asks how to create an account or expresses an intention to register, the chatbot should:

- Explain the registration process briefly.
- Display a clear interactive button such as **Create an Account**.
- Redirect the resident to the existing registration page when the button is selected.

Use the actual registration route already defined in the project. Do not invent a new route.

## C. Login assistance

When a resident asks how to log in or wants to access their account, provide a clear **Log In** button.

The button must navigate to the existing login page.

If the user is already authenticated, handle the situation appropriately instead of unnecessarily asking them to log in again.

## D. Complaint status lookup

Integrate the chatbot with the existing complaint tracking functionality.

When an authenticated resident asks about the status of a complaint:

1. Identify the authenticated resident.
2. Retrieve only complaint records that the resident is authorized to access.
3. If multiple complaints exist, ask which complaint they want to check or present a clear selection.
4. Display the relevant complaint reference or identifier, subject or appropriate description, current status, and available progress details.
5. Show the latest available updates and relevant dates when those details exist.
6. Explain the next step when the system has sufficient information to do so.

Use live database records and existing complaint-status logic. Do not return fabricated status information or hardcoded sample complaints.

If the resident is not logged in, ask them to log in before retrieving private complaint details.

The chatbot must never disclose another resident's complaint information, even if a user guesses a complaint identifier.

## E. Direct connection to an admin or barangay official

When a resident asks to speak with an official, contact an administrator, connect to a human, or sends a similar request, route the interaction to the existing admin messaging section.

The chatbot should:

- Recognize common expressions indicating that the resident wants human assistance.
- Check whether the resident already has an active connection or conversation.
- If an active conversation exists, inform the resident that they are already connected and can continue messaging.
- If no active conversation exists, use the existing messaging workflow to create or initiate the connection.
- Open or navigate to the appropriate messaging interface.
- Ensure the conversation appears in the admin messaging section so authorized staff can respond.
- Preserve conversation history and identify the associated resident correctly.

If the messaging system requires an administrator to accept or assign the conversation, communicate that state accurately. Do not falsely claim that an official is already connected when the system has only created a message request.

If the resident is not logged in, guide them through authentication before creating a private, account-linked conversation.

## F. Interactive buttons and conversation flow

Use context-sensitive buttons and quick actions where appropriate, including:

- Create an Account
- Log In
- Check My Complaint
- View Complaint Details
- Contact an Official
- Continue Existing Conversation

Only display buttons relevant to the current conversation.

Buttons must perform real actions using the existing application routes, APIs, and data. They must not be decorative or simulate successful operations.

Provide useful fallback responses when the chatbot cannot understand a request. Allow the resident to return to the main assistance options without restarting the entire conversation.

### G. Chatbot security and testing

Verify that:

- Registration and login buttons navigate correctly.
- Complaint status results come from real records.
- Residents can access only their own authorized complaints.
- Unauthenticated users cannot retrieve private complaint details.
- Existing admin messaging connections are detected correctly.
- New conversations appear in the admin messaging section.
- Connection status is communicated accurately.
- Chatbot errors do not break the existing messaging or complaint workflows.

---

## 5. INTERACTIVE "OUR SERVICES" SECTION ON THE HOMEPAGE

### A. Add animated complaint-type previews

Locate the existing **Our Services** section on the homepage.

When a user hovers over an individual complaint type, display an animated popup or overlay containing additional information about that complaint type.

The popup should appear above the existing homepage content and clearly explain what the selected complaint category covers.

Use the existing complaint categories and their actual descriptions. Do not invent new complaint types or change the system's complaint classification rules.

### B. Background blur and animation

When a complaint-type preview opens:

- Display the selected complaint information prominently.
- Apply a smooth background blur or dimming effect to the content behind the popup.
- Animate the popup's appearance and disappearance.
- Use smooth transitions for opacity, scale, and position where appropriate.

- Maintain the existing color palette, typography, and overall visual identity.

The interaction should feel responsive and intentional, not distracting or excessively animated.

### C. Interaction and accessibility

Support desktop hover interactions and provide equivalent click or tap behavior on mobile and touch-enabled devices.

Also support:

- Closing the popup by clicking outside it when appropriate.
- Closing it with the Escape key.
- Keyboard navigation and visible focus states.
- Appropriate accessible labels and focus behavior.
- Reduced-motion preferences for users who disable animations.

Ensure the overlay does not permanently block the page or prevent users from interacting with other sections after it closes.

Do not redesign the entire homepage. Modify only the existing Our Services section and the components or styles necessary to support this interaction.

### D. Testing requirements

Verify that each existing complaint type opens the correct preview, the background effect works properly, animations remain smooth, mobile interaction works, and the rest of the homepage remains functional.

---

## 6. EDITABLE CASE-CLOSURE REPORTS AND DOCUMENTATION

### A. Generate a report when a case is closed or resolved

Extend the existing admin complaint-management workflow so that when an authorized administrator or assigned staff member closes or resolves a case, the system can generate an editable report document for that complaint.

The generated document should resemble an official case report and contain the information required to document the complaint's handling and resolution.

Reuse existing complaint records, status history, audit logs, messages, evidence, and staff information wherever available.

Do not generate a report containing fabricated events, dates, evidence, or personnel.

## B. Required report contents

The report should include the following sections when the corresponding information is available.

### 1. Barangay and report identification

- Barangay San Andres, Cainta, Rizal.
- Report title and report reference number.
- Complaint reference or case number.
- Date the report was generated.
- Current case status.

### 2. Initial complaint information

- Original complaint submission date and time.
- Complaint category.
- Complaint subject and description.
- Exact incident address.
- Complainant information according to the applicable anonymity and privacy rules.
- Relevant incident details and supporting information.

The report must preserve the original submission date rather than substituting the case closure date.

### 3. Complaint progress and chronological history

Include the complaint's recorded history in chronological order, with dates and times where available.

Document:

- Complaint receipt and initial processing.
- Status changes.
- Follow-up actions.
- Case assignment or reassignment.
- Mediation or other relevant proceedings, if applicable.
- Additional evidence submitted by the resident.
- Administrative actions and remarks.
- Messages or other recorded communications relevant to the case.
- Final resolution or closure details.

Every entry must use actual records from the system. If a required historical detail was never recorded, do not invent it.

**4. Audit trail and supporting evidence**

Include relevant audit history for changes made to the complaint, including:

- Supporting evidence added by the resident.
- Changes to complaint information.
- Status updates.
- Relevant administrative modifications.
- Action remarks entered by the administrator or assigned staff.
- Dates and times of recorded changes.
- The identity or authorized display name of the person responsible for each action, when available and permitted.

Include links, references, or embedded copies of supporting photographs and files when supported by the existing storage system and report format.

Preserve the original evidence and timestamps. Do not replace existing files or alter audit history during report generation.

**5. Personnel involved in handling and resolving the case**

Include the names and roles of the assigned or responsible personnel, as recorded by the system.

Document the people involved in handling the complaint and the relevant resolution process, together with their recorded actions and dates.

Where applicable, include mediation participants, assigned staff, responsible officials, and other authorized personnel who participated in resolving the case.

Do not automatically label a person as responsible for resolution if the system does not record that responsibility.

**6. Final resolution and closure**

Include:

- Final complaint status.
- Resolution or closure date and time.
- Recorded resolution details.
- Final remarks or administrative findings.
- Relevant supporting evidence.
- The identity and role of the staff member who recorded the closure, when available.

## C. Editable document output

Generate an editable document format, preferably DOCX, unless the existing project already uses another suitable editable document-generation method.

The document must contain actual complaint information and remain editable after download.

Use a consistent report layout with appropriate headings, tables, dates, evidence references, and page formatting. Keep the design professional and appropriate for barangay case documentation.

Provide an appropriate action in the admin interface to generate and download the report after the case is closed or resolved.

Avoid generating duplicate reports unnecessarily. If a report is regenerated after legitimate updates, handle document versions or regeneration consistently with the existing workflow.

## D. Email notifications and recorded messages

Extend the existing email-notification workflow so that when a complaint is updated, relevant action remarks or messages entered by authorized staff are included in the corresponding notification email when appropriate.

Requirements:

- Include the complaint reference.
- Identify the update or status change.
- Include the relevant recorded remarks or message.
- Include the update date when available.
- Use the existing email templates, sending service, and notification architecture.
- Send notifications to the appropriate authorized recipients according to the existing rules.

Do not expose internal administrative notes, confidential information, masked-only information, or private third-party data to unauthorized recipients.

Do not send emails containing fabricated updates. Avoid duplicate notifications for the same event.

## E. Barangay officials in the report footer

At the bottom of the generated document, include the appropriate barangay officials' names and designations, following the structure used in the existing **Certificate to File Action** document.

The requested footer should accommodate:

- Barangay Captain.
- Barangay Secretary.
- Other relevant officials appearing in the existing Certificate to File Action format.

Inspect the existing document template or implementation to determine the exact layout and official designations.

Use the actual names maintained in the existing system or authorized configuration. Do not invent officials' names or assume that the currently logged-in administrator is the Barangay Captain or Secretary.

If the existing names are not stored in the application, identify the smallest appropriate configuration or data-management change needed to maintain them. Keep these values editable by authorized personnel where appropriate.

### F. Report generation security and testing

Verify that:

- A report can be generated for a closed or resolved complaint.
- The initial complaint date is correct.
- The report includes recorded progress and audit history.
- Evidence is correctly referenced or included when applicable.
- Administrative remarks and relevant messages appear appropriately.
- Staff names and roles match actual recorded assignments.
- The barangay officials' footer follows the existing Certificate to File Action structure.
- The document is editable after downloading.
- Anonymous complaints remain anonymous in generated reports wherever required.
- Sensitive information is not disclosed to unauthorized users.
- Report generation does not change or delete complaint records, evidence, or audit history.

---

# 7. IMPLEMENTATION WORKFLOW

Follow this implementation sequence.

### Phase 1 — Inspect the existing project

Before changing anything:

1. Inspect the repository structure.
2. Identify the frontend framework, styling setup, backend, database, authentication, and API structure.
3. Locate the complaint submission form and Step 3: Complainant Information.
4. Locate the incident-location form, existing Leaflet map, pinning functionality, and heatmap-area configuration.
5. Locate the existing chatbot, complaint tracking functions, and admin messaging section.
6. Locate the homepage's Our Services section.

7. Locate the case closure workflow, audit logging, email notifications, and any existing document-generation functionality.
8. Identify the current Certificate to File Action template or implementation.
9. Determine how the relevant modules communicate and which files actually need modification.

Do not assume filenames, routes, table structures, or libraries before inspecting the code.

## Phase 2 — Prepare a minimal implementation plan

Briefly list:

- Existing files and components that will be modified.
- Existing functions and services that can be reused.
- Necessary database or API changes.
- External services required, if any.
- Security and compatibility considerations.

Keep the plan focused on the six requested modification groups.

## Phase 3 — Implement incrementally

Implement the changes in small, logically grouped steps.

- Preserve existing code wherever possible.
- Avoid replacing entire components when a targeted change is sufficient.
- Reuse established styling conventions and shared components.
- Preserve existing authentication, authorization, routing, and data relationships.
- Add appropriate validation and error handling.
- Update related API and database logic only when necessary.
- Ensure that new functionality works with existing records and workflows.

## Phase 4 — Test the integration

Run the project's available checks, such as:

- Build and compilation.
- Linting and type checking.
- Existing automated tests.
- Relevant API and database tests.
- Manual testing of each modified user workflow.
- Regression testing of related existing features.

Test both successful and unsuccessful scenarios, including validation errors, missing data, unauthenticated access, unauthorized access, and external-service failures.

Do not claim that a test passed unless it was actually executed and its result verified.

### Phase 5 — Review the final changes

Before finishing:

- Review the modified files and explain why each change was necessary.
- Check that unrelated files and features were not changed unnecessarily.
- Verify that existing records remain intact.
- Confirm that privacy rules are enforced consistently across the UI, API, and generated documents.
- Identify any external API keys, credentials, boundary datasets, or official names that require configuration.
- Report any incomplete functionality or testing limitations honestly.

---

# 8. FINAL DELIVERY REQUIREMENTS

When the implementation is complete, provide:

1. A concise summary of the six modification groups implemented.
2. A list of the files changed and the purpose of each change.
3. A description of any database migrations or configuration changes.
4. A list of external services integrated and the setup required.
5. The tests executed and their actual results.
6. Any remaining limitations or tasks requiring manual configuration.

**FINAL REMINDER: This is an enhancement of an existing system, not a new development project. Preserve the existing application and make only the requested changes. Do not remove, replace, or redesign unrelated features. Prioritize correct integration, data privacy, actual functionality, and compatibility with the current codebase.**